Log for Random Forest

Student id: C00313459

Name: Amisha Das

1. Introduction

Using the Sloan Digital Sky Survey (SDSS) data, perform regression (prediction) task using random forest model to predict redshift values and then compare the results.

Later on, I performs randomizedsearchcv to see if I can make the model more accurate.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier,
RandomForestRegressor
from sklearn.metrics import accuracy_score, classification_report,
mean_absolute_error

sdss_path = "sdss.csv"
df = pd.read_csv(sdss_path)
```

About data-

Sloan Digital Sky Survey (SDSS) is a sky survey that collects magnitude values in 5 bands (u,g,r,i and z) and redshift values for stars, quasars and galaxies.

```
df.info()
df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 118070 entries, 0 to 118069
Data columns (total 18 columns):
 #   Column     Non-Null Count   Dtype
---  ------     --------------   -----
 0   objid      118070 non-null  float64
 1   ra         118070 non-null  float64
 2   dec        118070 non-null  float64
 3   u          118070 non-null  float64
 4   g          118070 non-null  float64
 5   r          118070 non-null  float64
 6   i          118070 non-null  float64
 7   z          118070 non-null  float64
```

```
 8   run         118070 non-null  int64
 9   rerun       118070 non-null  int64
 10  camcol      118070 non-null  int64
 11  field       118070 non-null  int64
 12  specobjid   118070 non-null  float64
 13  class       118070 non-null  object
 14  redshift    118070 non-null  float64
 15  plate       118070 non-null  int64
 16  mjd         118070 non-null  int64
 17  fiberid     118070 non-null  int64
dtypes: float64(10), int64(7), object(1)
memory usage: 16.2+ MB
```

	objid	ra	dec	u	g	r	i
0	1.237650e+18	183.531326	0.089693	19.47406	17.04240	15.94699	15.50342
1	1.237650e+18	183.598370	0.135285	18.66280	17.21449	16.67637	16.48922
2	1.237650e+18	183.680207	0.126185	19.38298	18.19169	17.47428	17.08732
3	1.237650e+18	183.870529	0.049911	17.76536	16.60272	16.16116	15.98233
4	1.237650e+18	183.883288	0.102557	17.55025	16.26342	16.43869	16.55492

	z	run	rerun	camcol	field	specobjid	class	redshift	plate
0	15.22531	752	301	4	267	3.722360e+18	STAR	-0.000009	3306
1	16.39150	752	301	4	267	3.638140e+17	STAR	-0.000055	323
2	16.80125	752	301	4	268	3.232740e+17	GALAXY	0.123111	287
3	15.90438	752	301	4	269	3.722370e+18	STAR	-0.000111	3306
4	16.61326	752	301	4	269	3.722370e+18	STAR	0.000590	3306

	mjd	fiberid
0	54922	491
1	51615	541
2	52023	513
3	54922	510
4	54922	512

```
features = ['u', 'g', 'r', 'i', 'z']
```

2. Performing regression and classification tasks

Regression will predict redshift Classification will predict object type

Adding choice to perform classification or regression. For the purpose of this notebook so far, I am going with classification

```
task = "classification"
```

Classification: maps the three features stars, galaxy and QSO as 0,1 and 2 respectively and assigning 100 decision trees. Uses RandomForestClassifier for classification.

Regression: dropping rows with missing redshift values and using RandomForestRegressor for regression with 100 trees.

```
if task == "classification":
    # convertinng class labels to numeric values
    df['class'] = df['class'].map({'STAR': 0, 'GALAXY': 1, 'QSO': 2})
    df = df.dropna(subset=['class'])  # drops the missing labels
    target = 'class'
    model = RandomForestClassifier(n_estimators=100, random_state=42)
elif task == "regression":
    df = df.dropna(subset=['redshift'])  # ddrop missing labels
    target = 'redshift'
    model = RandomForestRegressor(n_estimators=100, random_state=42)

X = df[features]
y = df[target]

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

model.fit(X_train, y_train)
```

```
RandomForestClassifier(random_state=42)
```

```
y_pred = model.predict(X_test)

if task == "classification":
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print(classification_report(y_test, y_pred))
elif task == "regression":
    print("Mean Absolute Error:", mean_absolute_error(y_test, y_pred))
```

```
Accuracy: 0.8927331244177183
              precision    recall  f1-score   support

           0       0.86      0.81      0.84      5644
           1       0.93      0.96      0.94     14442
           2       0.79      0.77      0.78      3528
```

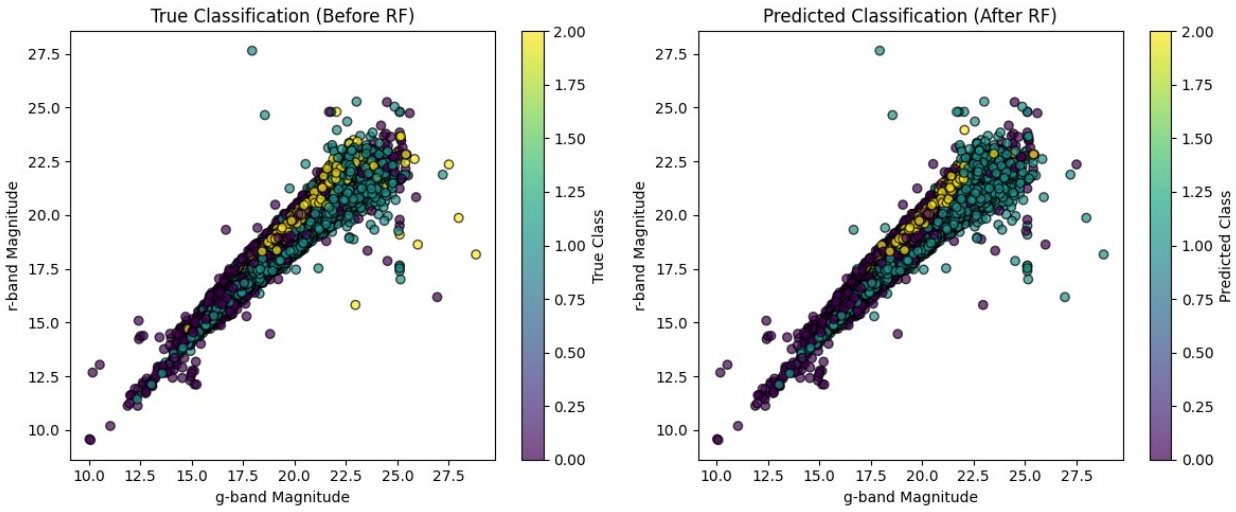
```
    accuracy                           0.89     23614
   macro avg       0.86      0.84      0.85     23614
weighted avg       0.89      0.89      0.89     23614
```

```
import matplotlib.pyplot as plt
```

Scatter plots before and after predictions

```
# before classification
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.scatter(X_test["g"], X_test["r"], c=y_test, cmap="viridis",
edgecolor='k', alpha=0.7)
plt.colorbar(label="True Class")
plt.xlabel("g-band Magnitude")
plt.ylabel("r-band Magnitude")
plt.title("True Classification (Before RF)")

# after classification (RF predicted labels)
plt.subplot(1, 2, 2)
plt.scatter(X_test["g"], X_test["r"], c=y_pred, cmap="viridis",
edgecolor='k', alpha=0.7)
plt.colorbar(label="Predicted Class")
plt.xlabel("g-band Magnitude")
plt.ylabel("r-band Magnitude")
plt.title("Predicted Classification (After RF)")
plt.tight_layout()
plt.show()
```



Confusion matrix for predicted classification from features/class where:

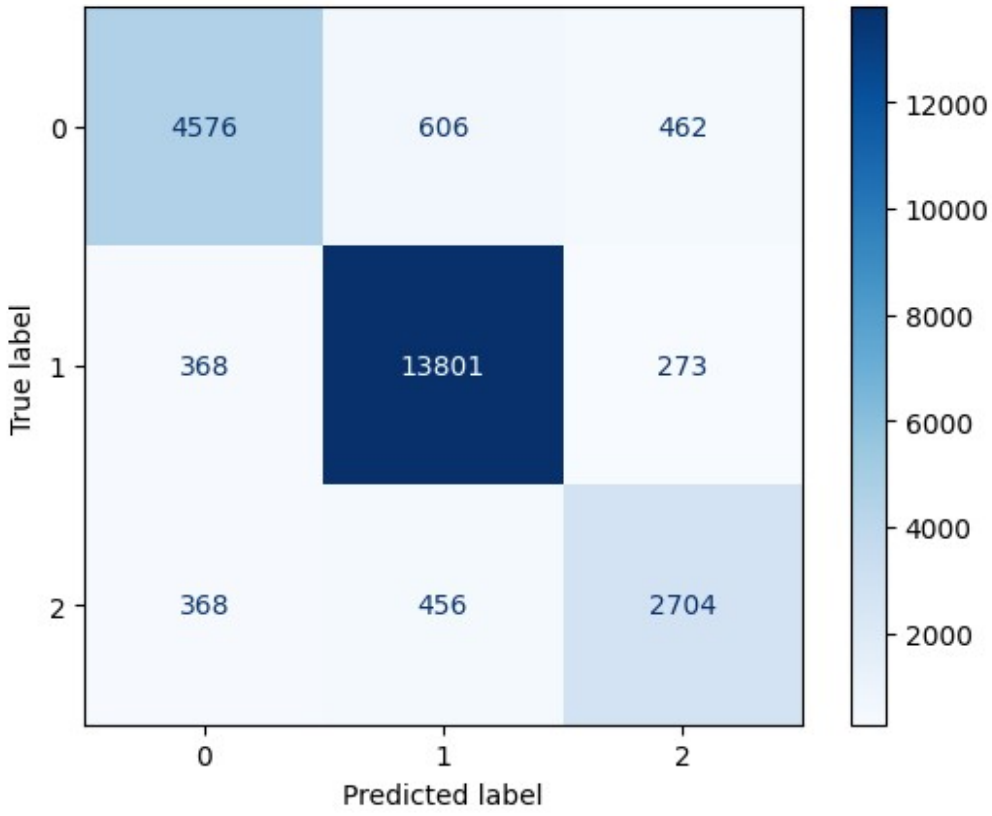
- 0- star
- 1- galaxy

- 2- quasar

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm=confusion_matrix(y_test,y_pred)

disp=ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap="Blues", values_format="d")
plt.show()
```



Conclusion

Classification results are good at 89% accuracy and a nice diagonal in the confusion matrix.

Increasing accuracy using randomizedsearchcv

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.ensemble import RandomForestClassifier
```

Using randomized search to descide which paramters are best for the random forest model to increase accuracy. Please check log for detailed rationale behind choosing parameters.

NOTE: Took around taking a very long time (8 hours estimated!) to grid_search to finish with 3 folds (243 fits)

LESSON: RF is computationally expensive. maybe reducing option from 3 to 2 would help or testing only a subset.

```
param_dist = {
    'n_estimators': [50, 100, 300],
    'max_depth': [10, 20],  # Prevent deep trees for faster results
    'min_samples_split': [2, 5, 10],  # Control overfitting
    'min_samples_leaf': [1, 2, 4]  # Keep best minimum leaf options
}



random_search = RandomizedSearchCV(
    RandomForestClassifier(random_state=42),
    param_distributions=param_dist,
    n_iter=20,
    cv=2,  # faster cross-validation
    n_jobs=-1,  # Use all CPU cores
    verbose=2,
    random_state=42  # Ensures reproducibility
)

random_search.fit(X_train, y_train)
```

```
Fitting 2 folds for each of 20 candidates, totalling 40 fits

RandomizedSearchCV(cv=2,
estimator=RandomForestClassifier(random_state=42),
                   n_iter=20, n_jobs=-1,
                   param_distributions={'max_depth': [10, 20],
                                        'min_samples_leaf': [1, 2, 4],
                                        'min_samples_split': [2, 5,
10],
                                        'n_estimators': [50, 100,
300]},
                   random_state=42, verbose=2)
```

```
print("Best Parameters:", random_search.best_params_)
```

```
Best Parameters: {'n_estimators': 300, 'min_samples_split': 5,
'min_samples_leaf': 1, 'max_depth': 20}
```

Retraining model:

Model is being retrained by the above parametres and will be compared with previous performance.

```
from sklearn.ensemble import RandomForestClassifier


best_params = {'n_estimators': 300,
```

```
                'min_samples_split': 5,
                'min_samples_leaf': 1,
                'max_depth': 20}


final_model = RandomForestClassifier(**best_params, random_state=42)
final_model.fit(X_train, y_train)
```

```
RandomForestClassifier(max_depth=20, min_samples_split=5,
n_estimators=300,
                       random_state=42)
```

```
print("Training complete with best parameters.")
```

```
Training complete with best parameters.
```

```
from sklearn.metrics import accuracy_score, classification_report
```

Predicting on test sets

```
y_pred = final_model.predict(X_test)


accuracy = accuracy_score(y_test, y_pred)
print(f"Final Model Accuracy: {accuracy:.4f}")

print("Classification Report:")
print(classification_report(y_test, y_pred))
```

```
Final Model Accuracy: 0.8910
Classification Report:
              precision    recall  f1-score   support

           0       0.86      0.80      0.83      5644
           1       0.92      0.96      0.94     14442
           2       0.79      0.77      0.78      3528

    accuracy                           0.89     23614
   macro avg       0.86      0.84      0.85     23614
weighted avg       0.89      0.89      0.89     23614
```

Conclusion

Negligible improvement. Need to work more on this.

3. Log

```
log = pd.read_csv("log.csv")
pd.set_option('display.max_colwidth', None)
log
```

```
                                                  Description  \
0                                                Added SDSS data
1                                Divided data into 80-20 ratio
2            Implemented classification and regression functions
3             Tested classification model, achieved 89% accuracy
4   Added comparative scatter plot for true vs predicted results
5                      Increased number of trees (n_estimators)
6                      Tuned hyperparameters using GridSearchCV
7                 Switched to RandomizedSearchCV for efficiency
8                    Reduced cross-validation folds from 3 to 2
9                          Retrained model with best parameters
10                              Results didn't improve much

   Level of Difficulty Time Spent    Change From  \
0                 Easy     10 min            NaN
1                 Easy     10 min            NaN
2               Medium     30 min            NaN
3               Medium    40 mins            NaN
4                 Easy     20 min            NaN
5               Medium     30 min  100, 300, 500
6                 Hard  1-2 hours            NaN
7                 Hard     1 hour   GridSearchCV
8               Medium    30 mins      3-fold CV
9               Medium     1 hour            NaN
10                 NaN        NaN            NaN


Changed To
0
Loaded SDSS dataset
1                                                  Split dataset
into training (80%) and testing (20%)
2                                    Added separate functions for
classification and redshift prediction
3
Initial model accuracy at 89%
4
Scatter plot visualization
5
Testing different tree counts
6                                                         Tested
243 fits with 3-fold cross-validation
7                                          RandomizedSearchCV
(testing 20 combinations instead of all 54)
8
2-fold CV
9   Used best_params={'n_estimators': 300, 'min_samples_split': 5,
'min_samples_leaf': 1, 'max_depth': 20}
```

```
10
No significant accuracy gain
```

References, Acknowledgements and Appendices

- Chatgpt for findign data set, ideas and debugging especially with randomizedsearchcv
- SDSS for data set, ideas and code debugging
- https://www.kaggle.com/code/ktrinh/sdss-classification-with-random-forests-99-2/notebook